

Code Explanation

Testing DLT Pipeline

This test suite is designed to validate the functionality of a DLT (Delta Live Tables) pipeline. The pipeline appears to be responsible for creating and managing several tables related to customer data and order information. The tests cover the creation and verification of these tables.

Step 1: Importing Necessary Modules and Setting Up the Test Environment

The test file begins by importing the necessary modules, including `pytest` for testing, `SparkSession` from `pyspark.sql` for creating a Spark session, and various other modules for data type definitions and mocking.

```
import pytest
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType, DateType
import datetime
import tempfile
import os
import sys
from unittest.mock import patch, MagicMock
```

Step 2: Creating a Mock DLT Module

To isolate the dependencies and make the tests more efficient, a mock DLT module is created. This mock module mimics the behavior of the actual DLT module by providing implementations for its key functions.

```
class MockDLT:
    def table(self, name=None, comment=None, table_properties=None):
        def decorator(func):
            return func
        return decorator

    def expect_or_fail(self, name, condition):
        def decorator(func):
            return func
        return decorator

    def read(self, table_name):
        if table_name == "customer":
            return self.mock_customer_df
        elif table_name == "order":
            return self.mock_order_df
        elif table_name == "ordersummary":
            return self.mock_ordersummary_df
        else:
            raise ValueError(f"Unknown table: {table_name}")
```

Step 3: Setting Up the Test Fixtures

The test suite uses several fixtures to set up the necessary environment for the tests. These include creating a Spark session and patching the DLT module with the mock implementation.

```

@pytest.fixture(scope="module")
def spark():
    return SparkSession.builder
        .appName("TestDLTPipeline")
        .master("local[1]")
        .config("spark.sql.warehouse.dir", tempfile.mkdtemp())
        .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
        .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
        .getOrCreate()

@pytest.fixture(scope="module")
def patch_dlt():
    with patch.dict("sys.modules", {"dlt": mock_dlt}):
        yield mock_dlt

```

Step 4: Preparing Mock Data

The `setup\_mock\_data` fixture is used to create sample data for the customer, order, and ordersummary tables. This data is used to test the DLT pipeline functions.

```

@pytest.fixture(scope="module")
def setup_mock_data(spark, patch_dlt):
    customer_schema = StructType([
        StructField("CustId", IntegerType(), True),
        StructField("Name", StringType(), True),
        StructField("EmailId", StringType(), True),
        StructField("Region", StringType(), True)
    ])

    customer_data = [
        (1, "John Doe", "john@example.com", "North"),
        (2, "Jane Smith", "jane@example.com", "South"),
        (3, "Bob Johnson", "bob@example.com", "East"),
        (4, "Alice Brown", "alice@example.com", "West")
    ]

    patch_dlt.mock_customer_df = spark.createDataFrame(customer_data, customer_schema)
    # ... similar setup for order and ordersummary data

```

Step 5: Testing the DLT Pipeline Functions

The test suite includes several test functions to validate the DLT pipeline. For example, `test\_customer\_table` tests the `customer` function by verifying the count and columns of the resulting DataFrame.

```

def test_customer_table(spark, patch_dlt, setup_mock_data):
    from src.dlt_pipeline import customer

    with patch('src.dlt_pipeline.spark', spark):
        result_df = customer()

```

```
assert result_df.count() == 4
assert "CustId" in result_df.columns
assert "Name" in result_df.columns
assert "EmailId" in result_df.columns
assert "Region" in result_df.columns
```

Step 6: Verifying the Order Table Functionality

The `test\_order\_table` function tests the `order` function, checking not only the count and columns but also the calculation of the `TotalAmount` column.

```
def test_order_table(spark, patch_dlt, setup_mock_data):
    from src.dlt_pipeline import order

    with patch('src.dlt_pipeline.spark', spark):
        result_df = order()

        assert result_df.count() == 4
        assert "OrderId" in result_df.columns
        # ... other column assertions

        row = result_df.filter("OrderId = 101").first()
        assert row["TotalAmount"] == row["PricePerUnit"] * row["Qty"]
```